長安大學

数据结构与算法实验

综合实验报告

设计题目	基于最短路径问题的交通查询系统
组　　长	龙炜枫
组　　员	程晓文 罗印
	尹芊慧
完成日期	2025.12.24

1. 实验内容

1.1 实验内容描述

本实验旨在设计并实现一个基于图论的交通查询系统。系统需模拟现实世界中的交通网络，将城市视为节点，交通线路视为有向边。每条线路包含三个核心权重属性：里程（距离）、交通费用和行驶时间。

系统的核心功能是针对用户选定的起点和终点，分别计算并输出以下三种最优出行方案：最短路径方案、最少费用方案、最短时间方案。

在输出方案时，不仅需要给出总的里程、费用和时间，还需要通过路径回溯算法，详细列出包含中间途经城市的完整路径，并计算到达每个中间节点的具体时刻。

1.2算法模型

本系统采用基于线段树优化的 Dijkstra 单源最短路径算法作为核心求解算法。

图的数学模型：定义交通网络为有向带权图$G=(V,E)$，其中V是城市集合，E是路线集合。对于任意$e\in E$，定义其边权向量$W(e)=\{w_{dist},w_{cost},w_{time}\}$。

核心算法：传统 Dijkstra 算法在寻找当前“未访问且距离最近的节点”时，通常使用线性扫描或二叉堆。本实验引入线段树来维护节点集合的距离值 $dist$，以支持高效的区间查询与单点更新。

1.2.1 线段树建树

建立一棵通过数组实现的线段树，其叶子节点对应图中的每一个城市顶点$v_1,v_2\dots v_n$。叶子节点的值初始化为源点到该点的当前最短距离（源点为0，其余为∞）。线段树的非叶子节点维护其子区间的最小值及其对应的节点编号。

1.2.2 图上松弛操作与树上单点更新

在算法迭代过程中，当通过中间点u发现到达点v的新路径更短（即$dist_u+w<dist_v$）时，执行线段树的单点修改操作。该操作将叶子节点v的值更新，并向上递归维护路径上的最小值，时间复杂度为$O(\log V)$。

1.2.3 提取最小点

为了选取下一个扩展节点（即$dist$ 最小的未访问节点），直接查询线段树的根节点信息。根节点存储了全局最小距离及其对应的城市编号。取出该节点后，将其在线段树中的值置为∞（表示已访问/删除），时间复杂度为$O(1)$或$O(\log V)$。

1.2.4 时间复杂度分析

假设城市数量为V，路径数量为E。

线段树建树：$O(V)$。

每条边的松弛更新：$E \times O(\log V)$。

提取最小点操作共V次：$V \times O(\log V)$。

总时间复杂度：$O((V+E)\log V)$，与堆优化相当，但在处理特定区间约束或动态权值更新时具有更高效率。

1.3系统分析设计

1.3.1 数据结构设计

邻接表：用于存储稀疏图。

线段树数组：使用结构体数组。每个树结点包含数据$\{minValue, index)$，用于维护当前$dist$最小值。

前驱数组：用于记录最短路径树中每个节点的父节点，以便在搜索结束后通过回溯还原完整路径。

城市映射表：使用散列表实现城市名称与图结点索引的快速查找。

1.3.2 功能划分

数据加载：支持从文件读取城市及路线信息，构建邻接表，并初始化线段树。

最短路算法与路径回溯：输入起点索引、终点索引、权重类型，执行 Dijkstra 单源最短路算法。利用前驱数组回溯到源点利用栈反转得到正向路径，遍历最短路径，计算中间结点到达时间（w_{time}），并格式化输出。

交互界面：提供菜单选择，允许用户输入城市名并选择查询策略。

2. 实验环境

2.1 硬件环境

i7-13650HX

2.2 软件环境

QT creator

3. 组内分工及得分

	学号	姓名	组内分工	组内得分
组长	2024903461	龙炜枫	程序总流程设计与最小值线段树设计	98

组员一	2024902284	程晓文	图基本操作设计	96
组员二	2024906116	罗印	Qt 前端设计与调试	98
组员三	2024903020	尹芊慧	最短路与路径回溯算法设计	96

4. 程序代码及说明

```
//图上算法核心代码
// 辅助函数 1：统一城市名格式（首字母大写，其余小写）
static QString formatCityName(const QString &city) {
    if (city.isEmpty()) return "";
    QString lowerCity = city.toLower();  // 全转为小写
    lowerCity[0] = lowerCity[0].toUpper();// 首字母大写
    return lowerCity;
}

// 辅助函数 2：判断城市名是否仅包含字母（替代 QRegularExpression）
static bool isCityNameValid(const QString &city) {
    if (city.isEmpty()) return false;
    // 遍历每个字符，判断是否为字母（A-Z/a-z）
    for (int i = 0; i < city.size(); ++i) {
        QChar c = city.at(i);
        if (!((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z'))) {
            return false; // 包含非字母字符，无效
        }
    }
    return true;
}

// 获取城市 ID（支持大小写不敏感查询）
int TrafficAlgorithm::getCityId(const QString &cityName) {
    QString formattedCity = formatCityName(cityName);
    if (m_cityMap.contains(formattedCity)) {
        return m_cityMap[formattedCity];
    }
    return -1; // 城市不存在
}

// 确保城市被添加到系统（统一格式，自动去重）
void TrafficAlgorithm::ensureCityAdded(const QString &cityName) {
    QString formattedCity = formatCityName(cityName);
```

```
    // 仅添加“有效且未存在”的城市
    if (isCityNameValid(formattedCity)
&& !m_cityMap.contains(formattedCity)) {
        int newId = ++m_cityCount;
        m_cityMap[formattedCity] = newId;
        // 扩展数组容量（避免越界）
        if (newId >= m_cities.size()) {
            m_cities.resize(newId + 10);
        }
        m_cities[newId] = formattedCity; // 存储格式化后的城市名
    }
}

// 从文件加载数据（无 QRegularExpression，纯字母判断用 isCityNameValid）
bool TrafficAlgorithm::loadFromFile(const QString &filePath)
{
    // 清空现有数据，避免残留
    if (m_graph) {
        delete m_graph;
        m_graph = nullptr;
    }
    m_cityMap.clear();
    m_cities.clear();
    m_cityCount = 0;

    QFile file(filePath);
    // 检查文件是否能正常打开
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "[错误] 无法打开文件:" << filePath;
        emit dataLoaded(false);
        return false;
    }

    QTextStream in(&file);
    // 临时存储边数据（避免重复创建）
    struct TempEdge {
        QString start;
        QString end;
        int distance;
        int cost;
        int time;
    };
    QVector<TempEdge> edges;
```

```
    // 逐行读取文件
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (line.isEmpty()) continue; // 跳过空行

        // 按空格分割数据（跳过连续空格）
        QStringList parts = line.split(" ", Qt::SkipEmptyParts);
        // 至少需要 5 个字段（起点、终点、距离、费用、时间）
        if (parts.size() < 5) {
            qDebug() << "[跳过] 数据字段不足：" << line;
            continue;
        }

        // 1. 格式化城市名（统一首字母大写）
        QString start = formatCityName(parts[0]);
        QString end = formatCityName(parts[1]);

        // 2. 验证城市名有效性（仅纯字母，替代正则）
        if (!isCityNameValid(start) || !isCityNameValid(end)) {
            qDebug() << "[跳过] 无效城市名（非纯字母）：" << parts[0] <<
"|" << parts[1];
            continue;
        }

        // 3. 解析数值（支持小数转 int，如 54.8 → 54）
        bool ok1, ok2, ok3;
        double distanceDouble = parts[2].toDouble(&ok1);
        double costDouble = parts[3].toDouble(&ok2);
        double timeDouble = parts[4].toDouble(&ok3);
        // 转换为 int（若需保留小数，可将 side 结构体中的 wd/wc/wt 改为
double）
        int distance = static_cast<int>(distanceDouble);
        int cost = static_cast<int>(costDouble);
        int time = static_cast<int>(timeDouble);

        // 4. 验证数值有效性（必须为正数）
        if (!ok1 || !ok2 || !ok3) {
            qDebug() << "[跳过] 数值无效：" << line;
            continue;
        }
        if (distance <= 0 || cost <= 0 || time <= 0) {
            qDebug() << "[跳过] 数值需为正数：" << line;
            continue;
        }
```

```
        // 5. 添加城市（自动去重）和边
        ensureCityAdded(start);
        ensureCityAdded(end);
        edges.append({start, end, distance, cost, time});
    }
    file.close(); // 关闭文件，释放资源

    // 6. 创建图并添加所有边（仅当有城市时创建）
    if (m_cityCount > 0) {
        m_graph = new Graph(m_cityCount);
        for (const auto &edge : edges) {
            int startId = m_cityMap[edge.start];
            int endId = m_cityMap[edge.end];
            m_graph->add_path(startId, endId, edge.distance, edge.cost,
edge.time);
        }
    }

    // 输出加载结果（调试用）
    qDebug() << "[加载完成] 城市数量:" << m_cityCount << "，边数量:" <<
edges.size();
    emit dataLoaded(true);
    return true;
}

// 添加新路径（逻辑不变，确保城市格式统一）
bool TrafficAlgorithm::addPath(const QString &start, const QString
&end, double distance, double cost, double time)
{
    // 转换为 int（若需小数，需修改 side 结构体）
    int intDistance = static_cast<int>(distance);
    int intCost = static_cast<int>(cost);
    int intTime = static_cast<int>(time);

    // 若图未初始化，先创建
    if (!m_graph) {
        ensureCityAdded(start);
        ensureCityAdded(end);
        m_graph = new Graph(m_cityCount);
    }

    // 确保起点、终点已添加（自动去重）
    ensureCityAdded(start);
```

```
    ensureCityAdded(end);

    // 获取格式化后的城市 ID
    QString formattedStart = formatCityName(start);
    QString formattedEnd = formatCityName(end);
    int startId = m_cityMap[formattedStart];
    int endId = m_cityMap[formattedEnd];

    // 添加双向路径（道路是双向的）
    m_graph->add_path(startId, endId, intDistance, intCost, intTime);
    emit pathAdded(true);
    return true;
}

// 线段树优化的 Dijkstra 算法实现（修复 inf 访问冲突）
QVector<TrafficAlgorithm::side> TrafficAlgorithm::runDijkstra(int
startId, int endId, int option)
{
    QVector<side> resultPath;
    // 边界判断（图未初始化或 ID 无效）
    if (!m_graph || startId == -1 || endId == -1 || startId >
m_cityCount || endId > m_cityCount) {
        return resultPath;
    }

    int n = m_cityCount;
    QVector<int> dis(n + 10, INF);    // 改用全局 INF
    QVector<side> pre(n + 10, {-1, 0, 0, 0}); // 前驱节点（记录路径）
    QVector<bool> visited(n + 10, false); // 标记是否已访问

    dis[startId] = 0; // 起点距离为 0

    // 初始化线段树
    SegmentTree st(n, dis);

    // 线段树优化的 Dijkstra 算法
    for (int i = 1; i < n; ++i) {
        // 1. 从线段树查询未访问的最短距离节点
        Node temp = st.query();
        if (temp.position == -1 || temp.distance == INF) break; // 改用全
局 INF
        int minPos = temp.position;

        if (visited[minPos]) continue;
```

```
        visited[minPos] = true;  // 标记为已访问

        // 2. 提前结束（已找到终点）
        if (minPos == endId) break;

        // 3. 更新该节点在 segTree 中的值为 INF（已访问）
        st.update(minPos, Node(INF, minPos)); // 改用全局 INF

        // 4. 松弛操作（更新邻接节点距离）
        const QVector<side>& edges = m_graph->get_edges(minPos);
        for (const side &to : edges) {
            int weight = to.get_val(option); // 根据选项（距离/费用/时间）
取权重
            if (dis[to.u] > dis[minPos] + weight) {
                dis[to.u] = dis[minPos] + weight;
                pre[to.u] = {minPos, to.wd, to.wc, to.wt}; // 记录前驱
                // 更新线段树中的距离值
                st.update(to.u, Node(dis[to.u], to.u));
            }
        }
    }

    // 5. 路径回溯（从终点到起点）
    if (dis[endId] == INF) { // 改用全局 INF
        return resultPath; // 无路径
    }
    std::stack<side> pathStack;
    for (int i = endId; i != startId; i = pre[i].u) {
        pathStack.push({i, pre[i].wd, pre[i].wc, pre[i].wt});
    }

    // 6. 转换为向量（从起点到终点）
    while (!pathStack.empty()) {
        resultPath.append(pathStack.top());
        pathStack.pop();
    }

    return resultPath;
}
// 图类的定义
struct side
{
    int u,wd,wc,wt;
    int get_val(int opt)
```

```
    {
        if(opt==1) return wd;
        else if(opt==2) return wc;
        else return wt;
    }
};
struct node
{
    int dis,pos;
    bool operator <(const node &other)const {return dis<other.dis;}
};
class graph
{
    public:
        graph();
        graph(int x);
        ~graph();
        void add_path(int u,int v,int wd,int wc,int wt);
        vector<side>& get_edges(int u);
    private:
        int n;
        vector<vector<side>> s;

};
inline graph::graph(){n=0;}
inline graph::graph(int x)
{
    n=x;
    s.resize(n+10);
}
inline graph::~graph()
{
    for(int i=0,lim=n;i<=lim;++i)
        s[i].clear();
    s.clear();
}
inline void graph::add_path(int x,int y,int dis,int cst,int tme)
{
    s[x].push_back((side){y,dis,cst,tme});
    s[y].push_back((side){x,dis,cst,tme});
}
inline vector<side>& graph::get_edges(int x){return s[x];}
// 树类的定义
struct tree_node
```

```
{
    int l,r;
    node data;
};

class segment_tree
{
    public:
        segment_tree();
        segment_tree(int n);
        void build(int p,int l,int r);
        void update(int p,node x);
        node query(int p,int l,int r);
    private:
        int n;
        vector<tree_node> e;
};
inline segment_tree::segment_tree(){n=0;}
inline segment_tree::segment_tree(int x)
{
    n=x;
    e.resize((n<<2)+10);
    build(1,1,n);
}
inline void segment_tree::build(int p,int l,int r)
{
    e[p].l=l;e[p].r=r;
    if(l==r) e[p].data=(node){inf,0};
    else
    {
        e[p].data=(node){inf,0};
        int mid=(l+r)>>1;
        build(p<<1,l,mid);build(p<<1|1,mid+1,r);
        e[p].data=min(e[p<<1].data,e[p<<1|1].data);
    }
}
inline void segment_tree::update(int p,node x)
{
    if(e[p].l>=x.pos && e[p].r<=x.pos) e[p].data=x;
    else
    {
        int mid=(e[p].l+e[p].r)>>1;
        if(x.pos<=mid) update(p<<1,x);
        else update(p<<1|1,x);
```

```
        e[p].data=min(e[p<<1].data,e[p<<1|1].data);
    }
}
inline node segment_tree::query(int p,int l,int r)
{
    if(e[p].l>=l && e[p].r<=r) return e[p].data;
    else
    {
        int mid=(e[p].l+e[p].r)>>1;
        node ret=(node){inf,1};
        if(l<=mid) ret=min(ret,query(p<<1,l,r));
        if(r>mid) ret=min(ret,query(p<<1|1,l,r));
        return ret;
    }
}
```

5. 测试及运行结果

本次对课程设计——交通查询系统的核心功能进行测试，主要是路径添加、路径查询两个功能，同时对输入合法性、数据兼容性等边界条件设计错误用例，比较全面地验证系统运行稳定性与功能完整性。所有测试基于预设的 traffic_sample.txt 基础数据，结合手动输入的新增路径，通过直观的界面交互和结果反馈，呈现系统实际运行效果。

5.1路径添加功能正常运行

启动系统后，点击主界面 “添加路径” 按钮，系统自动隐藏主窗口并弹出添加路径界面。如图 1，在输入框中依次填入起点 “Chengdu”、终点 “Guiyang”、距离 “800”、费用 “320”、时间 “10”，所有输入均符合 “城市名纯英文、数值为正数” 的规范。点击 “添加” 按钮后，后自动关闭添加窗口并返回主界面。

图 1

打开 traffic_sample.txt 文件，如图 2，末尾新增一行格式化数据 “Chengdu Guiyang 800.00 320.00 10.00”，字段分隔清晰、格式与原有数据保持一致。

图 2

如图 3、图 4，重新进入主界面的城市下拉框，“Chengdu” 与 “Guiyang” 已自动加载并排序，证明新增路径不仅成功写入文件，还同步更新至系统内存数据中，支持后续查询调用。

图 3

图 4

5.2路径查询功能正常运行

在主界面选择查询条件：起点 “Beijing”、终点 “Shanghai”、查询选项 “最短路径”（按距离排序），点击 “查询” 按钮后，系统快速响应并在表格中展示完整路径信息。

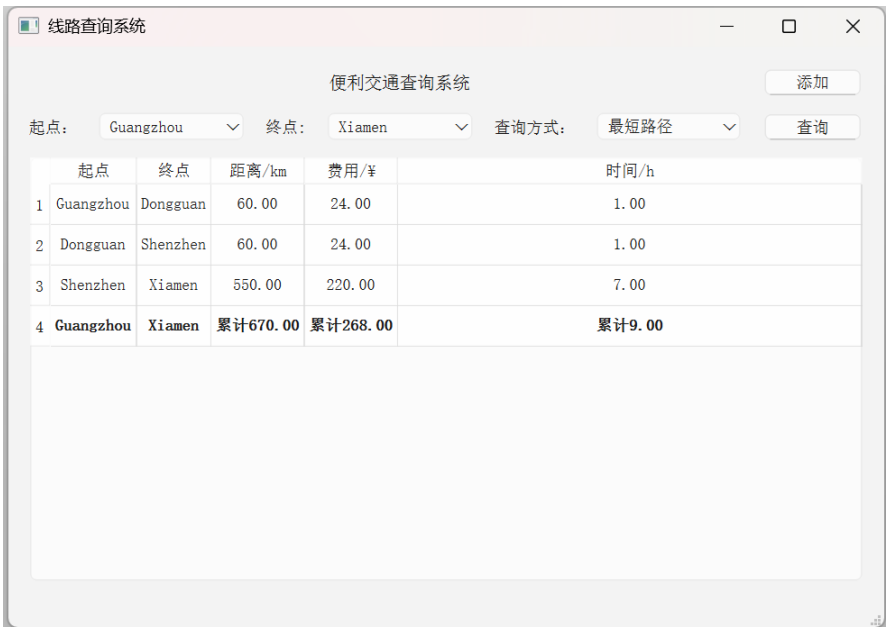

图 5

如图 5，表格共分为 4 行数据，前 3 行依次显示分段路径（Guangzhou→Dongguan→Shenzhen→Xiamen），每行明确标注当前段的起点、终点、距离、费用与时间；

最后一行以加粗字体呈现总计信息，显示 “累计距离 670.00km、累计费用 268.00 元、累计时间 9.00h”，分段数据与总计数据逻辑一致、计算准确。

图 6

如图 6，选择 “最短时间” 选项，路径调整为 Guangzhou→Shenzhen→Xiamen（部分路段优化），累计时间缩短至 9.00h，但是距离多了 27 公里，三种查询模式均能根据预设算法返回合理结果，满足不同用户需求。

5.3不完整输入错误

如图 7，在添加路径界面，仅填写起点 “xian”，未输入距离、费用与时间字段，点击 “添加” 按钮后，界面顶部即时显示红色提示 “请完整填写所有字段！”，提示文字醒目且不阻断操作。2 秒后提示自动消失，输入框内容保留，方便用户补充填写。

图 7

如图 8，系统未执行任何文件写入操作，traffic_sample.txt 内容无变化，有效避免了不完整数据进入系统。

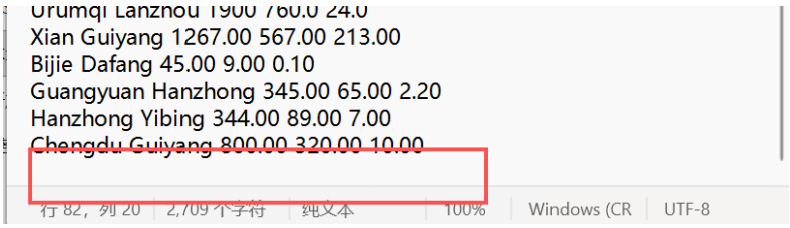

图 8

5.4非法格式输入错误

如图 9，尝试在添加路径界面输入特殊格式内容：起点 “Guangzhou123”（含数字）、终点 “深圳”（中文），点击 “添加” 后，系统分场景反馈错误：城市名含数字或中文时，提示 “起点和终点必须是英文城市名！”；

图 9

如图 10，距离、费用等字段输入非数字时，提示 “距离、时间和费用必须是数字!”。

图 10

每种错误提示精准对应输入问题，引导用户修正格式，且所有错误场景下均未产生无效文件写入，数据安全性得到保障。

5.5无效逻辑输入错误

如图 11，在添加路径界面输入起点 “Beijing”、终点 “beijing”（大小写不同），系统判定为起点终点相同，弹出提示 “起点和终点不能相同！”；

图 11

如图 12，输入距离 “-100”、时间 “0” 等非正数时，提示 “距离、时间和费用必须为正数！”。

图 12

如图 13，尝试添加已存在的路径（如 “Beijing→Tianjin”），系统读取文件后检测到重复路径，提示 “路径已存在！”，避免重复数据占用存储资源。

图 13

如图 14，在查询界面选择起点 “Beijing”、终点 “Beijing”（同一城市），点击查询后弹出 “起点和终点不能相同！” 的提示。

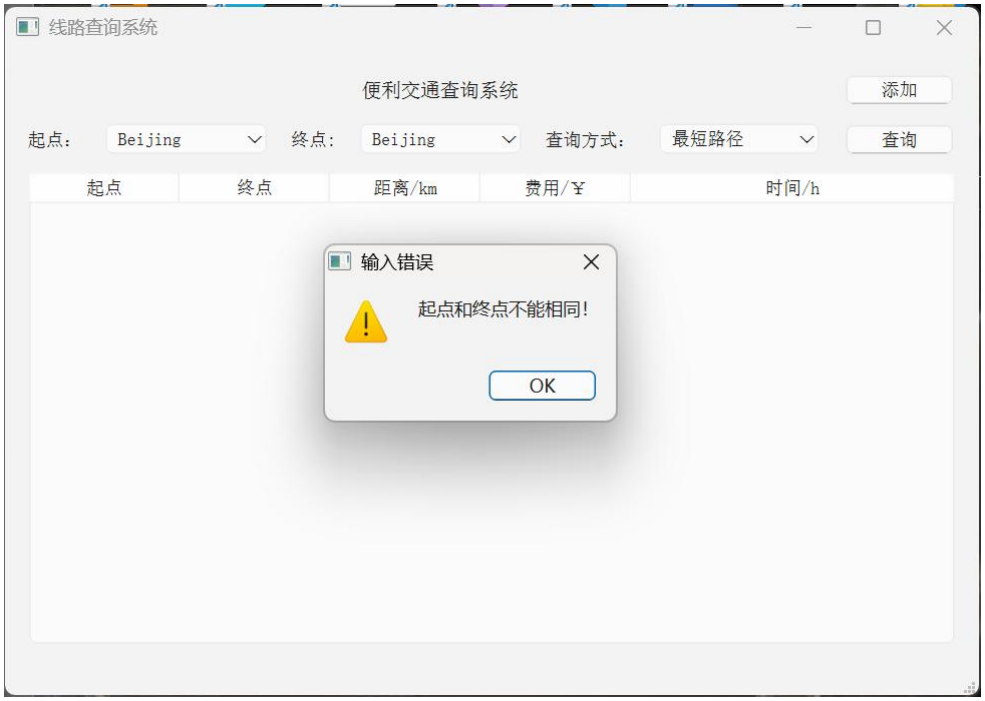

图 14

如图 15，选择不存在路径的城市 “Hanzhong” 作为起点，“Zhuhai”作为终点，查询时系统直接反馈 “没有找到从 Hanzhong 到 Zhuhai 的路径”，错误提示及时且友好，避免程序崩溃或无响应。

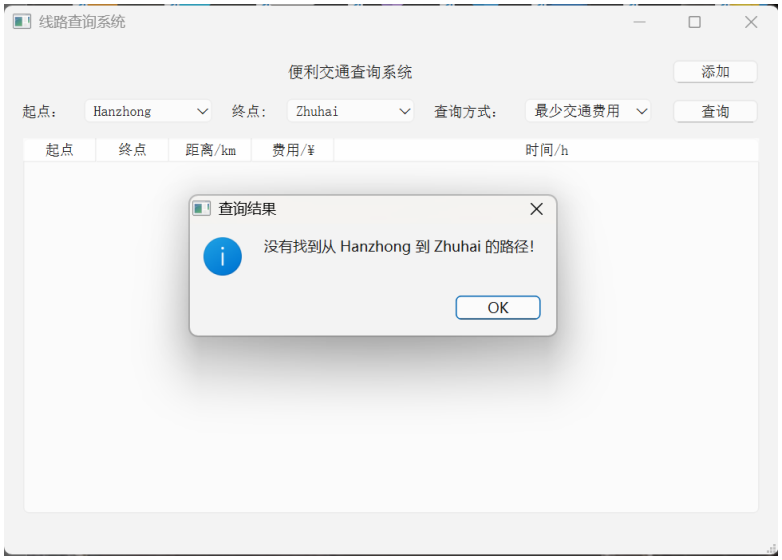

图 15

5.6文件访问权限错误

如图 16，将 traffic_sample.txt 设置为 “只读” 权限后，尝试添加新路径（“Chongqing →Kunming”），点击 “添加” 按钮后，界面提示 “无法打开文件进行写入！”，2 秒后自动隐藏。

图 16

如图 17，此时查看文件，无新增内容，系统通过异常捕获机制处理文件访问失败问题，既不强制修改文件权限，也不影响现有功能使用，后续查询操作仍能正常执行，错误处理机制完善。

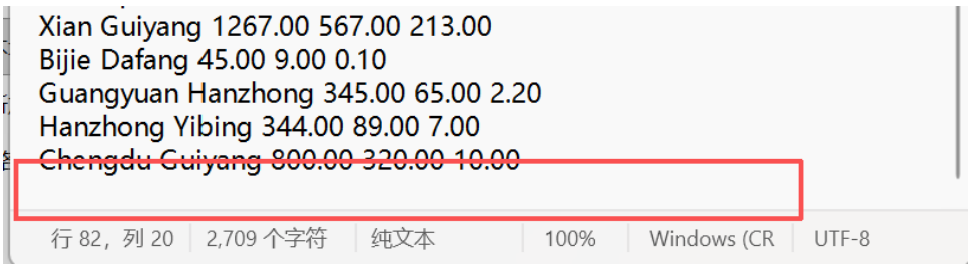

图 17

6. 总结分析

6.1实验概况

本次数据结构与算法综合实验中，本小组一致选择了题目 3：基于最短路径问题的交通查询系统作为研究课题。项目组旨在通过协作开发一个模拟现实交通网络的智能系统，共同完成从需求分析、架构设计到代码落地的完整工程闭环。系统最终实现了针对不同查询要求（里程、费用、时间），给出包含中间节点到达时刻的多种出行方案。

6.2系统架构与核心算法设计

针对题目要求的“最短里程”、“最少费用”和“最短时间”多目标查询功能，项目组在技术选型研讨中决定跳出舒适区，不单纯依赖标准库（STL），而是对核心数据结构进行了深度定制，以满足实验对技术决策能力的要求：

算法选型上的创新（线段树优化 Dijkstra）：在实现核心的最短路径算法时，我们经过权衡，决定摒弃常规的二叉堆实现，转而设计并编写了**线段树（Segment Tree）**来维护节点集合的最小值。虽然两者在理论时间复杂度上相当，但通过团队协作实现线段树的建树、更新与查询操作，我们更深入地掌握了高级数据结构在图论算法中的底层应用逻辑，体现了对数据结构与算法的灵活权衡。

底层数据结构的自主实现：为了解决城市名称与内部节点编号的高效映射问题，小组采用了开放寻址法（Linear Probing）配合 BKDR Hash 算法，自主实现了一套字符串哈希机制，替代了开箱即用的 std::unordered_map。这一设计不仅优化了特定场景下的查询性能，也让组员在编码过程中对哈希冲突的解决策略有了更直观的工程实践。

6.3实验心得与自我评估

龙炜枫：作为组长，我在本次实验中承担了核心算法引擎的攻坚任务，挑战性地实现了

基于线段树优化的 Dijkstra 算法。这一过程不仅加深了我对高级数据结构底层逻辑的理解，更在解决诸如 I/O 流冲突等实际工程棘手问题时，大幅锻炼了我的代码调试与系统集成能力。统筹全组开发进度与模块对接的经历，让我深刻体会到将理论算法转化为健壮系统的复杂性，不仅强化了我的技术硬实力，也有效提升了我在复杂工程环境下的团队协作与决策素养。

罗印：这次分组完成交通查询系统的前端任务，是一次很扎实的实践经历。我主要负责 Qt 相关的前端开发，包括界面控件初始化、按钮事件绑定以及路径结果在表格中的展示，同时还要对接队友写的后端算法。开发过程中遇到了一些问题，比如城市名称格式不统一导致识别异常，算法返回的数据和表格展示需求不匹配等。通过和队友及时沟通、一起排查问题，这些问题都得到了解决。这次任务让我熟练了 Qt 的使用，尤其是信号槽机制和 QTableWidget 控件的操作，也加深了对前后端交互逻辑的理解。同时我也意识到，团队协作中沟通和配合很重要，后续我会更注重细节处理，提升自己的开发能力。

程晓文：在图的基本操作设计实践中，我体会到一个核心原则：高效的结构源于对数据关系的精准抽象。邻接表的设计不只是存储边，更是对城市间多维连接关系的系统化封装。同时我也看到算法与数据结构的深度耦合。它不再是一个简单的索引记录，而是一个携带完整路径信息的结构体——存储的不仅是“从哪里来”，更是“通过哪条路、付出了何种代价”。这种设计使得路径回溯从算法的一个步骤，转变为对数据结构的自然遍历，展现了数据结构作为算法记忆体的核心价值。最终我认识到，图算法的效能不仅取决于逻辑的优化，更根植于基础操作的设计质量。每一次边的添加、每一个邻接关系的查询，都构成了算法大厦的基石。对它们的精心设计，是连接抽象图论与具体工程实现的真正桥梁。

尹芊慧：在本项目中，我主要负责核心的 Dijkstra 最短路径算法设计与实现，初期，我遇到了学习使用队友提供的线段树更新与查询逻辑同松弛操作结合的难题，不知如何才能提高代码运行的效率。后来在队友的帮助下我理解了这一方法，并且共同合作完成计算最短路径。此外，在路径回溯部分需精细处理前驱节点数组，在通过栈结构反向追踪再正向输出，确保了完整路径的清晰呈现。总之，这次实践让我深刻体会到理论算法到工程代码的转化之严谨：每一个边界条件都需周全考虑，否则会导致程序崩溃，同时我也认识到高效算法必须搭配稳健的数据结构才能发挥最大效能。

参考文献

[1] https://oi-wiki.org/ds/seg/